

THURSDAY, SEPTEMBER 24 · AFTERNOON

4 Persistent instructions and memory · 5 Connectors and plugins · 6 Agent harnesses

Kerry Back

This afternoon

Persistent instructions

Standing rules the agent reads every turn, and tasks written down once.

Connectors and plugins

Tools on somebody else's server, and folders of skills other people published.

Agent harnesses

Claude Code is one program around the model. What the others decided differently.

Four hands-on blocks. By six you will have written a skill, connected to data you do not hold, and installed plugins other people wrote.

4 · PERSISTENT INSTRUCTIONS AND MEMORY

The repeated instruction

This morning you typed the same four things into every prompt: describe the data first, show the code, report row counts, reconcile a second way.

Each of those is a standing instruction about how you want work done. Typing them into each prompt makes them optional, and one of them gets left out at four in the afternoon.

Everything arrives as one block of text

Each turn, the harness assembles the request from parts you can control:

PART	SET BY	WHEN IT IS SENT
System prompt	the tool	every turn
Tool definitions	the tool and your config	every turn
<code>CLAUDE.md</code>	you	every turn
Skill descriptions	you	every turn, one line each
A skill's body	you	only when that skill is used
The conversation	you and the tools	growing, every turn

The cost of always

A file read on every turn is paid for on every turn — in money, in latency, and in the model's attention.

CLAUDE.md

Short and always present. Standing rules that apply to everything you do in that folder.

A skill

One line always present — its description. The body arrives only when the description matches what you asked for.

A **CLAUDE.md** that runs to three thousand words is paid for on every turn, including the turns none of it applies to.

What belongs in each

Standing rules

- How you want numbers handled
- What the folders hold
- Conventions everything here follows
- Anything you have corrected twice

Not standing rules

- Steps for one particular task
- Anything longer than a page
- Background that can be looked up
- A tool you use once a month

Write your standing rules

EXERCISE 5 · 30 MINUTES

- 1 Create `~/CLAUDE.md` in your container.
- 2 Write the four rules from this morning in your own words, and two more of your own.
- 3 Start a new conversation and give it a task that would break one of the rules.
- 4 Change a rule and watch the behavior change.

Ask it to name the rules it is following. If it cannot, the file is not being read.

SKILLS

A skill is a folder

```
country-report/  
  SKILL.md          the instructions  
  references/       longer material, read only when needed  
  scripts/          code the skill runs rather than regenerates
```

SKILL.md opens with two lines of metadata and then plain prose:

```
1 ---  
2 name: country-report  
3 description: Build the monthly revenue-by-country report from the  
4   order spreadsheets. Use when asked for the country report, the  
5   monthly revenue rollup, or revenue by market.  
6 ---
```

The description is the trigger

DOES NOT FIRE

“Helps with reporting.”

FIRES

“Build the monthly revenue-by-country report from the order spreadsheets.
Use when asked for the country report, the monthly revenue rollup, or revenue by market.”

The description is the only part of a skill always in front of the model, and it decides whether the rest is ever read. Write the words the person asking would use.

What goes in the body

- 1 What the task produces, stated first.
- 2 Where the inputs are, by path.
- 3 The steps in order, including the checks.
- 4 What to do when something is missing.
- 5 What the output looks like when it is right.

Prose, not pseudocode. The reader is a model that already knows Python and does not know your files.

INSTRUCTIONS

The code is written fresh each time.
Same intent, a different
implementation, different bugs.

A SCRIPT

The code is written once and tested.
The skill says when to run it and what
the arguments mean.

Scripts beat instructions

Three ways to invoke

	HOW	WHEN
Slash command	<code>/country-report</code>	you know exactly what you want
By name	“use the country report skill”	you are half sure
Automatically	“build me the March rollup”	the description matched

The third only happens if the description was written for it.

Already in your container

SKILL	WHAT IT DOES
<code>xlsx</code>	builds and edits Excel workbooks, with formulas and charts
<code>docx</code>	the same for Word, including templates
<code>pptx</code>	the same for PowerPoint, including an existing deck
<code>finance-data</code>	pulls prices, fundamentals and macro series
<code>web-app</code>	scaffolds an app you can serve on port 8000
<code>skill-creator</code>	writes and tests a new skill

Open one. It is the same kind of file you are about to write.

Write a skill

EXERCISE 6 · 50 MINUTES

- 1 Pick something you do at work more than once a month, with inputs and a shape.
- 2 Run `/skill-creator` and work through it, answering in your own vocabulary.
- 3 Put a real input file in the container and run the skill on it.
- 4 In a fresh conversation, ask for the task in the words a colleague would use.
- 5 Fix the description until it fires.

Step 4 is the test. A skill that fires only when you name it has not been described yet.

5 · CONNECTORS AND PLUGINS

The problem a standard solves

Every tool wants to reach every data source. Without a standard, that is one integration per pair.

$$N \times M$$

INTEGRATIONS WITHOUT A PROTOCOL

$$N + M$$

WITH ONE

A server written for nothing in particular is reachable from every client that speaks the protocol.

What a connector actually sends

- 1 The client connects to the server and asks what it offers.
- 2 The server returns a list of tools: each with a name, a description in English, and the parameters it takes.
- 3 That list goes into the model's context alongside its built-in tools.
- 4 The model asks for one by name. The client calls the server and puts the result back in the conversation.

The model does not know a connector from a built-in tool. Both are entries in the same list.

A tool definition

```
1 {  
2   "name": "run_sql",  
3   "description": "Run a read-only SQL query against the Northwind  
4                   database and return the rows. SELECT only.",  
5   "input_schema": {  
6     "type": "object",  
7     "properties": {  
8       "query": {"type": "string", "description": "A SELECT statement"}  
9     },  
10    "required": ["query"]  
11  }  
12 }
```

The description is what the model reads to decide whether to use it. A badly described tool is an unused tool.

Where the call is made from

In Claude Cowork, and in most setups a firm will hand you, the machine the agent's shell runs on has no route to the internet. The connector works anyway.

AT THE SHELL

```
1 curl https://api.stlouisfed.org/ ...
```

Refused. Everything leaving that machine passes a proxy that allows a short list of addresses, and that is not one of them.

THROUGH THE CONNECTOR

```
1 fred_get_observations("UNRATE")
```

Rows come back. Same machine, same second, same model.

Because your machine never places the call. The client end sits outside it, calls FRED itself, and hands the rows back into the conversation. Your key goes with it and is never on the machine to steal.

A narrower door, not a hole

OPEN THE NETWORK INSTEAD

Allow the machine to reach `stlouisfed.org` and it can send anything to any address there, in any form it likes.

LEAVE IT SHUT, ADD A CONNECTOR

It can call `run_sql` with a `SELECT`. That is the entire new capability.

The connector is the smaller of the two. That is why the network stays off and the connector goes on, and not the other way around.

It costs something. The connector reaches out from outside the sandbox holding your credentials, so a sentence in a file it reads can ask for that call. Cutting the machine off from the network did not cut off the connector. Sunday morning.

Servers you can connect today

SERVER	ADDRESS	SIGN-IN
Northwind orders	<code>northwind-mcp.rice-business.org/mcp</code>	none
Energy data (EIA)	<code>eia.rice-business.org/mcp</code>	none
Economic data (FRED)	<code>fred.kerryback.com/mcp</code>	none

Same three servers, reachable from Claude Code in your container and from Claude Desktop on your laptop. Neither was written for either one.

Connect one

EXERCISE 7 · 40 MINUTES

- 1 Connect the Northwind server, and then one of the two data services.
- 2 Ask what tools each one offers, and read the descriptions.
- 3 Ask the Northwind server which countries generated the most revenue, and have it show the SQL.
- 4 Ask something that needs the connector and a file in your container at the same time.
- 5 Try to make the database server do something other than read.

Keep that number. Tomorrow you compute it yourself from spreadsheets and reconcile the two.

WHERE THE LIMIT LIVES

Read-only, enforced four ways

WHERE	WHAT REFUSES
The database	a role without write permission
The application	a whitelist of allowed operations
The connector	<code>run_sql</code> rejecting anything that is not a <code>SELECT</code>
The tool list	no write tool exists to call

The model is not consulted at any of them. That is the property that makes them worth having.

Instructions are not a limit

A RULE IN A PROMPT

“Never modify the database.”

Text in a conversation, competing with other text in the same conversation.

A TOOL THAT DOES NOT EXIST

There is nothing to call.

No wording reaches it, because nothing is being persuaded.

Everything you can talk it out of was a request, not a constraint.

Mail and calendar

The same shape applied to an inbox: an agent that reads everything and sends nothing.

It reads

Threads, calendar entries, attachments. Enough to tell you what is waiting and what it is about.

It drafts

Replies, parked where you can read them.

It does not send

No send tool, no delete tool. Sending stays a thing you do.

Why that split is the whole design

An agent reading your mail is holding two things at once: your private correspondence, and text written by people outside your organization.

Give it an outbound channel and those two facts combine into a third. We come back to this on Sunday, in detail.

INSTALLING PLUGINS

What you install

A plugin is a folder somebody else published. Installing it adds their work to your Claude in one step.

What it can carry

Skills, slash commands, subagents, hooks, and MCP servers.

What you get

Every file and script in it, without being told which ones to copy where.

Two commands

```
/plugin marketplace add kerryback/skills  
/plugin install finance-data@kerryback
```

The first points Claude at a public repository holding an index of plugins. The second installs one of them. `/plugin` on its own lists what you have.

A marketplace you can use today

`kerryback/skills` holds fifteen. Seven worth your time:

`finance-data` prices, fundamentals, SEC filings and macro series, as CSV

`elegant-pdf` branded flyers, programs and reports as print-quality PDF

`critique` parallel reviewers on work Claude produced, and what to do

`human-write` prose that does not read as machine-generated

`shoji2` Quarto slide decks in the theme these slides use

`voiceover` a narrated MP4 and transcript from a deck

`smithers` a read-only desk over your mail and calendar

Read it before you install

An installed plugin runs with your permissions, on your machine, against your files.

- 1 Open the repository and read the `SKILL.md` of anything you are about to install.
- 2 Check what it runs: a script, a command, an MCP server reaching outside.
- 3 Install into a container first when you are not sure.

Same question as Sunday morning's, asked earlier: what can it reach, and what can leave.

Install and try

EXERCISE 8 · 40 MINUTES

- 1 Add the `kerryback/skills` marketplace and list what is in it.
- 2 Install two plugins that look useful for your own work.
- 3 Run one of them on your own data or your own document.
- 4 Open its `SKILL.md` and read how it does what it does.
- 5 Say what you would change, and what you would have written differently.

You wrote one of these earlier. Step 4 is reading somebody else's.

	WHERE IT LIVES	WHO HAS IT
A prompt	in your head	you, when you remember
CLAUDE.md	one folder	anyone working in that folder
A skill	a file	you, everywhere, when it matches
A plugin	a repository someone published	anyone who installs it

The same instruction each time, moved somewhere that outlasts the conversation.

Four rungs

6 · AGENT HARNESSES

You have spent a day inside Claude Code. It is one answer to a design problem, and there are others — increasingly not from the model's maker.

Same model, different program

The model is a string in a config file. Everything you experience as “the tool” is the program wrapped around it: what it sends each turn, which tools it offers, how it asks permission, and where it runs.

The harness is the other five

You pick the model. Everything else on that list is supplied by the program around it.

Buying a harness — Claude Code, or one of the open ones — is buying somebody's answers to those five questions. The answers are not the same, and this year they stopped being the same in ways that show up in the output.

What actually differs

THE QUESTION IT ANSWERS

Model coupling	one vendor's models, or any provider you can point it at
Context	what gets sent each turn, and what happens when the window fills
Tool surface	four tools, or forty
Permissions	how it stops — approval rules, an OS sandbox, or a container
Extension	plugins, MCP servers, skills, instruction files
Where it lives	a terminal, an editor, a web page, or a library you import

Every one of these is a design choice somebody made and wrote down. None of them is the model.

The open field

Claude Code	Anthropic models only	the reference implementation
Codex	OpenAI models only	Rust; OS-level sandbox by default
OpenCode	75+ providers	the mature open alternative; per-agent permissions
DeepSeek Harness	any provider	everything is a plugin, including the loop
Pi	any provider	four tools and a very short prompt
Goose · OpenHands	any provider	neutral governance; container-first
Gemini CLI · Qwen Code	their own models	vendor CLIs on the same pattern

Everything but Claude Code is open source, MIT or Apache — you can read the loop. Two of those still only talk to their maker's models.

Thick or thin

CLAUDE CODE

Forty-plus tools. A layered system prompt with a cached boundary. Four fallback strategies when the context fills. Subagents that fork and run in the background.

The harness does a great deal on your behalf, and you did not write any of it.

PI

Four tools — read, write, edit, run a command. A system prompt under a thousand tokens. No subagents, no plan mode, no MCP.

Anything else, the agent writes for itself as an extension while you watch.

Databricks benchmarked both and found the thin one matched or beat the thick one at a fraction of the cost, because it sent roughly three times less context per turn.

The permission model

HARNESS	HOW IT STOPS YOU
Claude Code	rules you configure, checked before each call
Codex	the operating system — Seatbelt on macOS, Landlock on Linux
OpenHands	a Docker container the agent cannot leave
Cline	a human clicking approve, every time
OpenCode	per-agent tool permissions, granted once and remembered

The distinction from this morning, in a different form: a mode that asks you, versus a boundary that makes the call impossible. Two of these five are the second kind.

Why any of this is on the slide

A benchmark number is a model and a harness together. Change the harness and the number moves.

- 1 The same model scores differently under two harnesses on the same benchmark.
- 2 Papers report the model and omit the harness.
- 3 Removing most of an agent's tools has been shown to raise its score, not lower it.
- 4 So a leaderboard tells you less than it appears to.

When somebody tells you a model can or cannot do something, the useful question is what it was wearing at the time.

Everything today has been about setting the agent up. Tomorrow you point it at data and find out whether you can trust what comes back.

Tomorrow